linope

路漫漫其修远兮，吾将上下而求索。

本篇是关于 Linope 内核的技术报告，旨在用尽量通俗易懂的方式介绍 Linope 的设计思想、开发范式以及其中的一些技术细节。

摘要

随着越来越多的公司和开发者开始开发 Agent，一个问题逐渐变得值得讨论：

Agent 系统的内核和传统操作系统内核有什么区别？

带着这个问题，我开发了 Linope Agent 内核。Linope 希望借鉴传统操作系统的设计思想，同时结合 Agent 系统的特点，探索一种不同于传统 Agent 框架的开发范式。

文章主要从两个方面展开：

- Linope 的设计思想；
- Linope 的 Agent 开发范式。

我一直在思考，一个 Coding Agent 和一个医疗顾问 Agent，如果使用同一套内核，它们之间的区别究竟在哪里？

一个 Coding Agent，换一些工具，修改一些提示词和知识库，再通过 MCP 接入网站、软件或其他服务，是不是就可以转变成一个医疗顾问 Agent？

今天，我们可以使用 Codex、Claude Code 等 Agent 完成很多不同类型的任务。这些 Agent 并不会因为任务从编码变成搜索、写作或分析，就彻底改变内部的协调逻辑。开发者通常只需要接入新的工具，增加新的 Skill，或者调整知识来源，就可以赋予 Agent 新的能力。

这得益于 LLM 综合能力的不断提升。过去，我们经常需要用大量代码控制 Agent 的工作流程；而现在，LLM 的规划能力、理解能力和创造能力越来越强，很多复杂任务已经可以交给模型自行拆解和推进。

DeepSeek Harness 的开源也进一步说明了这一点：一个 Agent 可以通过插件获得新的能力，而不需要为每个新任务重新编写一套 Agent。

因此，Linope 对 Agent 的基本定义是：

Agent = 内核 + 能力。

在 Linope 中，内核不实现具体的模型和工具。内核负责把外部能力装入一个平级能力进程。这个能力进程可以是 LLM，也可以是搜索工具、文件操作工具或其他服务，并且内核应具备一定的灵活性。

如果沿着这个思路继续设计，那么 Agent OS 内核就可以借鉴传统操作系统的设计方式，围绕进程、线程和资源进行管理，包括：

- 进程间通信；
- 进程生命周期；
- 线程启动和并发控制；
- 负载均衡；
- 资源管理；
- 上下文管理；
- 网络和沙盒管理。

这些能力在 Agent 系统中分别对应模型调用、工具调用、上下文读取、能力进程之间的通信，以及高并发任务下的资源调度。

我们需要注意的是：内核内部的能力主要负责协调外部能力进程，而不负责实现每一个具体能力。

所谓新的 Agent 开发范式

前面提到，Linope 的设计思想是：

Agent = 内核 + 能力。

Linope 所探索的新范式，并不只是把模型和工具放进几个独立进程，而是重新定义 Agent 的组成方式：

Agent 不再是一个内部写满业务流程的程序，而是一组由内核协调的平级能力进程。

在很多 Agent 框架中，Agent 通常以一个 Session 或一个 Agent 实例为中心。模型调用、工具调用、上下文拼接、压缩和任务循环，都围绕这个实例展开。开发者通过注册插件、工具和事件监听器，为这个 Agent 增加新的能力。

以deepseek-harness为例。它使用 Cordis 组织插件，使用 SessionEvent 保存会话事实，再由 surface 投影出模型能够看到的消息，最后由 Agent Loop 驱动 turn、step、模型请求和工具调用。

它的核心关系大致是：

代码块

```
插件系统
    ↓
Agent Loop
    ↓
SessionEvent 日志
    ↓
消息投影
    ↓
模型请求
```

这种方式的优点是会话容易持久化、重放和恢复。模型看到的内容都可以从 Session 日志中重新构造出来；上下文压缩时，也可以用一条摘要消息替换旧的消息范围，同时保留原始事件。

但这种方式的基本单位仍然是一个完整的 Agent 会话。工具、模型和上下文管理器通常作为插件挂在这个会话或运行时上，它们并不天然拥有独立的资源账本、地址空间和生命周期。

Linope 选择了不同的切入点。

在 Linope 中，内核不把模型、工具和上下文压缩器看成 Agent 内部的几个函数，而是把它们都看成能力进程：

- 一个模型进程代表推理能力；
- 一个搜索进程代表联网搜索能力；
- 一个代码执行进程代表执行能力；
- 一个上下文压缩进程代表上下文处理能力。

它们之间没有谁天然属于谁。内核只负责：

- 创建和回收能力进程；
- 管理进程持有的资源；
- 在能力进程之间传递请求；
- 选择当前仍然存活并且能够处理任务的进程；
- 管理 memory、tool、sandbox 和 network 等资源；
- 根据上下文压力和进程负载进行调度。

因此，Linope 与deepseek-harness的主要区别，是插件和能力所处的位置不同。

上文讲到过，在 deepseek-harness 中，插件主要扩展一个 Agent 运行时。插件通过 Cordis 注册服务、事件和能力，Agent Loop 负责推动整个会话继续执行。

在 Linope 中，能力本身就是内核可以调度的对象。模型和工具都拥有独立的进程身份，分别持有自己的资源租约和账本。模型可以通过内核提供的页面查看当前存活的能力进程，再决定调用哪一个进程。

这也解释了为什么 Linope 取消了父子进程和进程树的概念。

取消父子进程并不代表能力进程之间没有关系，而是代表这种关系不再由“谁启动了谁”决定。进程之间的关系通过资源、地址和通信建立，而不是通过进程树继承。

从开发方式上看，Linope 的变化是：

过去，开发者往往需要先设计一个 Agent 的工作流，再把模型、工具和上下文处理代码填入这个工作流。

在 Linope 中，开发者首先声明这个 Agent 需要哪些能力和资源，然后把每种能力实现为一个可以被内核启动和调用的进程。至于当前应该搜索、执行代码、询问另一个模型，还是读取历史上下文，则由模型结合当前页面进行选择，这也体现了我上面说的“内核应该具备灵活性。”随着模型综合能力的

逐渐提升，我们应该将越来越多的决策交给模型来处理，内核只是辅助模型，提供系统中协调调度的能力，而不是限制模型的思考。（参考文章：Anthropic：As models become more capable, agentic design will trend towards letting intelligent models act intelligently, with progressively less human curation.https://www.anthropic.com/research/effective-context-engineering-for-ai-agents ）

这并不意味着 Linope 完全没有流程。进程生命周期、权限检查、资源回收、通信和请求验证仍然由内核负责。变化在于，内核不再把具体业务流程写死。内核负责保证执行安全和资源可控，模型负责在能力之间进行选择和规划。

例如，一个 Coding Agent 和一个医疗顾问 Agent 可以使用同一套进程、资源和调度机制。两者的区别主要来自：

- 使用的模型；
- 挂载的工具；
- 使用的知识来源；
- system prompt 和 Skill；
- memory 的内容；
- sandbox 和 network 权限；
- 对应的安全策略。

当然，医疗场景不能只靠替换提示词完成。医疗知识来源、隐私保护、风险分级、人工升级和结果审计，都应该作为独立能力或资源加入系统。但这些变化不需要重新设计 Agent 的调度内核。

这就是 Linope 所说的新开发范式：

用资源声明描述 Agent 需要什么能力，用平级能力进程提供这些能力，用内核统一管理它们的生命周期、通信和权限，而不是为每一种 Agent 单独编写一套协调逻辑。

从这个角度看，deepseek-harness 更强调“如何可靠地保存和重建一次 Agent 会话”，而 Linope 更强调“如何把 Agent 拆解为一组可以隔离、共享和调度的能力进程”。

两者并不是谁替代谁，而是关注的层次不同：

代码块

```
deepseek-harness:
会话日志 → 消息投影 → Agent Loop → 模型请求

Linope:
能力进程 → 资源租约 → 指针空间 → 内核调度
```

Linope 所谓的“新”，核心不在于把 Agent 改成了几个进程，而在于改变了 Agent 的组织单位：

从“一个拥有插件的 Agent”，转变为“一个由内核协调的能力进程集合”。

为什么要把能力脚本视作进程

能力，指的是外部能力。创建一个外部能力进程，就代表给 Agent 赋予了某种能力。

例如：

- 创建一个模型进程，代表赋予 Agent 推理能力；
- 创建一个搜索进程，代表赋予 Agent 联网搜索能力；
- 创建一个文件工具进程，代表赋予 Agent 文件读写能力；
- 创建一个上下文压缩进程，代表赋予 Agent 上下文管理能力。

我之所以把外部能力脚本视作进程，是因为在传统操作系统中，进程是资源隔离的基本单位。

传统操作系统中的父子进程，通常承担几类作用：

1. 子进程启动时可以继承父进程的地址空间、文件描述符、当前目录和环境；
2. 进程组和会话可以把多个进程组织起来，统一发送信号和停止任务；
3. 父进程可以通过 wait 回收子进程的退出状态，内核也可以清理对应的进程表项。

这些机制适合传统操作系统中的程序执行，但 Agent 系统中的能力调用具有不同特点。

一个模型可能在运行过程中调用搜索工具，搜索工具又可能调用文件工具，随后模型还可能启动另一个模型处理子任务。如果把这些能力全部组织成固定的父子进程树，那么进程之间的关系会在启动时被固定下来，系统的灵活性会受到限制。

Linope 因此取消了父子进程和进程树的概念，让能力进程彼此平级。

这样设计有几个好处。

进程生命周期彼此独立

一个工具进程崩溃时，内核只清理这个工具进程，不会因为一个能力进程出错而影响其他进程。

模型进程仍然可以继续运行，其他工具进程也不需要全部退出。这种设计更符合 Agent 的实际运行方式：能力调用是动态发生的，某一次工具失败不应当自动摧毁整个 Agent。

资源隔离不会通过继承扩散

平级进程启动时，只会拿到内核明确授予它的信息，例如：

- 资源句柄；
- 资源类型；
- 调用参数；
- 当前能力所需要的配置。

它不会自动获得其他进程的资源实体编号，也不会因为某个进程是它的“父进程”就继承全部资源。

在环境变量方面，能力只获得它明确声明需要的密钥。模型能力声明了某个 API key，内核才会把对应密钥复制到进程环境中；其他敏感环境变量不会因为进程继承而泄露。

资源共享必须显式声明

平级进程不会因为属于同一个 Agent，就自动共享 memory 或其他资源。

默认情况下，同一个 scope 的 memory 也可以被编译成不同的独占资源。只有显式声明共享时，进程才会共同使用一个资源实体。

当前实现中，--share（资源共享的命令行） 针对的是已经声明的资源种类，例如 memory。模型本身不是 kernel/resource 中独立注册的资源类型，模型身份主要通过能力配置和进程设置传递。因此，模型能力进程的共享和 memory 资源的共享需要分别理解，不能简单地等同为同一种资源租约。

进程间调用面向当前存活进程

传统进程树中的关系通常在创建时确定，但 Agent 的工具调用是动态的。

模型可能先调用搜索工具，之后调用代码执行工具，再调用另一个模型。工具是否存在、是否空闲、是否仍然存活，都可能随着运行状态变化。

因此，Linope 的模型进程不会根据固定的父子关系寻找工具，而是通过内核页面查看当前可用的能力进程，再决定调用对象。

进程之间的调用关系由当前状态决定，而不是由进程树创建时的结构决定。

资源账本可以按进程结算

Linope 为每一个 pid 建立独立的资源记录。

租约记在对应进程的账本中：

- 进程启动时获得租约；
- 进程运行时使用资源；
- 进程退出时释放租约；
- 资源引用计数归零后，实体被卸载。

线程只是进程内部的执行方式，不单独建立资源账本。

如果按照进程树管理资源，那么停止一个节点之后，还需要决定整棵子树的资源租约应该归谁。资源账本会与“谁碰巧启动了谁”绑定在一起。

Linope 则把资源直接绑定到进程。memory 是进程使用的上下文资源，tool 是进程使用的工具连接，sandbox 是进程的执行空间。进程退出，这些授权就随之收回。

进程间的通信

如果能力进程彼此平级，那么进程之间就不能依靠父子关系自动传递数据。它们需要通过内核提供的通信机制交换任务和结果。

Linope 中的进程间通信主要解决三个问题：

- 一个进程如何向另一个进程发送任务；

- 一个进程如何获得另一个进程的结果；
- 发送者的资源和上下文如何避免泄露给接收者。

在模型调用工具的场景中，模型进程可以向工具进程发送一个任务，但工具进程不需要看到模型进程的完整 memory。接收方只得到当前任务和它自己拥有的资源。

工具执行结束后，结果再通过通信通道返回给模型进程。结果可以被写入模型自己的 memory，也可以作为下一页的一部分返回给模型。

在 Linope 的 `kernel/communication/channel.py` 中，通信被区分为两类。

`MemoryChannel` 主要用于向进程自己的 memory 写入和读取内容。它适合保存用户输入、模型结果、工具结果以及进程内部的上下文。

`ServiceChannel` 则表示一个已经由调用方打开的服务通道。它提供：

- `send`：向指定地址发送一条消息；
- `publish`：向某个主题发布消息；
- `receive`：接收发给当前地址的消息；
- `close`：关闭通道。

内核本身不负责决定使用哪种网络传输方式，也不会自动为所有能力进程创建通信连接。调用方需要先打开通道，再通过通道进行通信。

这意味着，通信机制和能力实现是分离的：

代码块

```
能力进程：
负责处理任务

通信通道：
负责传递任务和结果

内核调度器：
负责选择通信对象和检查访问权限
```

例如，模型进程不需要知道搜索工具的内部实现，只需要知道当前有一个可以处理搜索任务的能力进程。内核负责找到这个进程，并把请求通过通道交给它。

这种设计也为未来的远程通信留下了空间。当前两个能力进程可以运行在同一台机器上，通过本地通道通信；未来也可以将其中一个进程部署到另一台机器，通过 HTTP、RPC 等协议通信。

需要注意的是，进程间通信不等于共享 memory。进程可以通过通信传递一项任务，但不代表接收方自动获得发送方的上下文地址空间。任务内容、结果内容和 memory 访问权仍然是三个独立的问题。

线程：进程内部的执行单元

在 Linope 中，进程是资源隔离和资源结算的基本单位，线程则是进程内部执行任务的基本单位。

一个能力进程可能同时收到多个请求。例如，一个模型进程可能同时等待多个工具结果，一个搜索工具进程也可能同时处理多个搜索任务。

如果每一个请求都启动一个新的进程，会带来较高的启动成本，也会增加资源管理的复杂度。因此，Linope 使用线程在同一个能力进程内部并行处理任务。

线程主要承担以下作用：

- 从进程的输入通道中领取任务；
- 执行一次能力调用；
- 等待模型、工具或网络返回；
- 将结果提交给进程的输出通道；
- 在进程规定的并发上限内处理多个任务。

线程和进程之间的关系可以理解为：

```
代码块
1    进程：
2    拥有资源、句柄、账本和生命周期
3
4    线程：
5    使用进程已经获得的资源，执行具体任务
```

线程不会单独拥有 memory、sandbox 或 network 租约，也不会单独建立资源账本。一个进程内的多个线程共享这个进程已经获得的资源。这也符合了传统操作系统中对于进程和线程的规定。

如果某个线程执行失败，通常只影响当前请求；如果整个能力进程退出，那么该进程内部的线程都会停止，进程持有的资源也会一起释放。

因此，线程适合解决并发问题，进程适合解决隔离和生命周期问题。Linope 不把线程当作资源隔离边界，也不把线程当作独立能力。

把 Agent 看作服务

除了把 Agent 看作进程之外，还可以从另一个角度把 Agent 看作服务。

进程和服务并不是互相排斥的两个概念，而是处于不同层次：

```
代码块
1    进程：
2    Agent 在系统内部如何运行
3
4    服务：
```

```
5   Agent 如何被其他进程、Agent 或外部应用访问
```

一个 Agent 可以在本地作为一个能力进程运行，同时对外表现为一个服务。进程拥有 pid、资源租约和运行时状态；服务则拥有访问地址、调用协议、能力描述和生命周期接口。

把 Agent 看作服务之后，Agent 就不再只是一个被动等待模型调用的脚本，而可以成为一个被发现、被调用和被组合的系统组件。

例如，一个医疗 Agent 可以作为医疗问答服务被其他 Agent 调用；一个 Coding Agent 可以作为代码分析，编写服务被任务调度器调用；一个搜索 Agent 可以作为搜索服务被多个模型进程共同使用。一个足够通用的Agent或许可以由多个Agent服务构成，根据具体的业务需求去设计。那么未来的某一天，我们会不会像微服务一样按照业务需求拆分细化各种Agent，从而变成一种Agent版本的微服务？（这个猜想我将会在下面进行说明）

这种设计和当前 GitHub 上一些 Agent 项目的发展方向是一致的。

AgentScope Runtime 将 Agent 暴露为 Agent-as-a-Service，并提供流式 API、工具沙盒、可观测性以及本地、Kubernetes 和 Serverless 部署方式。它的设计重点不是把 Agent 当作一次函数调用，而是把 Agent 运行时包装成可以持续提供服务的应用。

A2A 项目 则进一步把 Agent 之间的通信抽象成开放协议。A2A 允许不同框架、不同公司和不同服务器上的 Agent 互相发现能力、协商交互方式，并通过 JSON-RPC over HTTP(S)、流式传输或异步通知完成任务协作。

A2A 特别强调 Agent 的内部状态可以保持不透明。调用方不需要知道另一个 Agent 使用什么模型、拥有什么内部 memory 或工具，只需要通过 Agent Card 了解它提供什么能力、如何连接以及支持什么交互方式。

这与 Linope 的设计存在一定的对应关系：

代码块

```
1   Linope 进程：
2   能力、资源、句柄、pid
3
4   Linope 服务：
5   地址、服务能力、通信协议、调用关系
6
7   A2A：
8   Agent Card、任务协议、远程 Agent 之间的协作
```

当然，这些项目并不是完全相同的实现。AgentScope Runtime 更关注 Agent 的生产部署，A2A 更关注不同 Agent 之间的互操作，而 Linope 关注的是内核层面的进程、资源和调度。

但它们共同说明了一件事：

> Agent 正在从一个只能被内部调用的程序，逐渐变成可以被其他系统发现和调用的服务。

从这个角度看，Linope 的能力进程可以成为服务的运行基础。一个服务可以由一个能力进程提供，也可以由多个能力进程共同组成。内核负责服务背后的进程生命周期、资源租约和负载均衡，服务接口则负责对外暴露能力。

服务间的通信

进程间通信主要发生在内核管理的能力进程之间，服务间通信则发生在两个独立服务之间。

二者的区别在于，服务间通信不一定发生在同一台机器上，也不一定共享同一个进程管理器。服务之间可能存在：

- 网络延迟；
- 服务发现；
- 身份认证；
- 权限控制；
- 请求超时；
- 重试；
- 任务状态查询；
- 流式输出；
- 异步回调；
- 版本兼容。

因此，服务间通信不能简单地看成“把进程间通信搬到网络上”。它需要更明确的服务契约。

一个服务至少需要说明：

- 自己是谁；
- 能够完成什么任务；
- 接受什么输入；
- 返回什么结果；
- 任务是否可以长时间运行；
- 是否支持流式结果；
- 如何查询任务状态；
- 如何取消任务；
- 需要什么权限。

Linope 当前的 `ServiceChannel` 已经提供了一个抽象方向：调用方通过 `send`、`publish` 和 `receive` 与服务通信，而具体使用本地通道、HTTP、RPC 还是其他传输方式，可以由外部实现决定。

这样，内核管理的是服务的调度和资源，通信层管理的是服务之间的数据传递。

HATEOAS 与 Linope 的超媒体链接

服务之间如何让调用方知道“当前可以做什么”，是服务设计中的一个重要问题。

一种常见做法是把所有接口写死在客户端中：

```
客户端知道：
/search
/tool
/ask
/start
/stop
```

这种方式的问题是，客户端必须提前知道所有操作。如果服务当前不可用、权限发生变化，或者服务增加了新的操作，客户端仍然可能按照旧的接口发起请求。

Linope 采用了另一种思路：让当前页面告诉模型或调用方当前有哪些可用动作。

在 `kernel/hypermedia` 中，页面由事实和链接组成。memory 页面可以包含：

- 当前 memory 的状态；
- 当前回合内容；
- 历史内容的地址；
- 可读取的 memory 链接；
- 可以调用的工具；
- 可以询问的模型；
- 可以启动的能力进程；
- 最终回答的入口。

`memory_page()` 负责生成 memory 相关链接，`model_page()` 则根据当前存活的进程和权限增加工具、模型以及启动能力的链接。

这是一种受 HATEOAS 启发的设计。

HATEOAS 是 REST 架构中的一个约束，全称是：

> Hypermedia As The Engine Of Application State

它的核心思想是：客户端不需要预先知道服务所有可能的状态转换，而是通过服务返回的超媒体链接发现下一步可以做什么。

相关参考文献主要有两个：

1.Roy Fielding 的博士论文中对 REST 架构的定义，尤其是“Hypermedia as the engine of application state”这一约束：
https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

2.RFC 8288《Web Linking》，它定义了 Web 链接、链接关系和目标资源之间的表达方式：
https://www.rfc-editor.org/rfc/rfc8288

需要说明的是，Linope 当前的 `kernel/hypermedia` 并不是一个完整的 REST 服务实现，也不是简单地把网页链接交给模型点击。它借鉴的是 HATEOAS 的核心思想：由当前资源状态动态提供下一步可执行的关系和入口。

例如，模型页面可能告诉模型：

代码块

```
memory/message/0
tool:search
ask:model-02
process start tools.ddgs
answer
```

模型不需要提前记住所有工具命令，只需要根据当前页面选择一个可用链接。

Linope 使用这种设计有几个原因。

第一，减少硬编码的调度逻辑

如果模型和客户端提前写死所有工具和进程，那么每增加一种能力，都需要修改模型提示词、客户端代码或调度逻辑。

通过页面提供链接，新能力可以由内核动态展示。模型只需要理解当前页面，不需要知道所有可能存在的能力。

第二，反映当前运行状态

某个工具进程可能已经退出，某个模型可能正在忙，某个能力可能没有被授予，某个服务也可能因为权限原因不能使用。这些状态可以直接影响页面中是否出现对应链接。页面展示的不是系统全部能力，而是当前时刻能够尝试的能力。

第三，把发现和执行分开

页面中的链接只是一个可执行入口，不是最终授权。

模型即使看到了某个地址，调度器仍然需要检查：

- 进程是否存在；
- 资源是否存在；

- 当前进程是否拥有对应权限；
- selector 是否位于 pointer space；
- 句柄是否仍然有效。

因此，HATEOAS 式页面负责告诉模型“可以尝试什么”，内核权限系统负责判断“是否真的允许”。

第四，适合服务化扩展

在本地进程模式下，链接可以指向一个能力进程、一个 memory 地址或一个内核命令。

在未来的服务模式下，链接也可以指向远程服务、任务资源或 A2A 操作。

例如，一个远程医疗 Agent 可以返回：

```
代码块
1   self
2   capabilities
3   task
4   status
5   cancel
6   result
```

调用方不需要知道服务内部是由几个进程组成的，只需要按照链接关系继续完成任务。这实现了客户端和服务端的一种解耦。

第五，减少上下文浪费

传统方式可能把所有工具说明、所有历史内容和所有可用命令都放入模型上下文。

Linope 的页面只提供当前相关的地址和动作。模型需要历史时，再读取对应的 memory 地址；需要工具时，再跟随工具关系。

这和上下文工程中的一个重要原则一致：

> 不要把所有可能的信息一次性放进上下文，而应该在需要时提供与当前决策相关的信息。

需要再次强调，链接不是安全授权。一个模型可以伪造字符串地址，但它不能因此获得资源。所有真正的读取和调用，仍然必须经过 Linope 内核中的 selector 检查、pointer space 检查和资源句柄检查。

Agent 服务化与未来的微服务尝试

上文提到：一个足够通用的Agent或许可以由多个Agent服务构成，根据具体的业务需求去设计。

当前的 Linope 主要运行在本地进程和内核调度模型中。未来，可以把一个能力进程包装成一个独立服务，让它具有：

- 独立的服务地址；

- 独立的版本；
- 独立的资源配额；
- 独立的部署位置；
- 独立的可观测性；
- 独立的升级周期。

此时，Agent 的调用关系可能从：

```
代码块
1    模型进程 → 内核 → 工具进程
```

扩展为：

```
代码块
1    模型服务 → 服务发现 → 工具服务
```

内核仍然可以负责资源和权限，但通信过程可以由 HTTP、RPC、A2A 或其他协议完成。

不过，把 Agent 服务化并不意味着简单地给现有进程增加一个 HTTP 接口。真正的微服务化还需要处理很多问题：

- 网络中断后任务如何恢复；
- 请求重复发送时是否会产生重复副作用；
- 服务版本如何兼容；
- 长时间运行的任务如何查询状态；
- 远程服务如何认证；
- memory 到底由哪一方拥有；
- 工具结果是否需要持久化；
- 服务出现故障时由谁负责重试；
- 不同服务之间如何传递 trace 和审计信息。

因此，更准确的说法是：

> Linope 将 Agent 拆解为平级能力进程，并通过资源、通信、服务描述和超媒体链接保留了向 Agent 微服务演进的可能性。

进程是服务在本机上的运行载体，服务是能力对外提供的访问边界。

二者可以同时存在：

代码块

```
Agent 服务
    ↓
一个或多个能力进程
    ↓
资源租约、线程、工具和 memory
```

这样，未来替换通信方式时，不必重新设计能力本身。只需要为能力增加新的服务提供者和传输实现。

Linope 的资源设计

资源管理是操作系统内核的重要部分。

传统操作系统中的资源通常包括：

- CPU；
- 内存；
- I/O 设备；
- 磁盘；
- 网络设备。

但 Linope 位于传统操作系统之上。CPU、GPU、真实的电脑内存和磁盘，仍然由传统操作系统管理。Linope 只通过宿主操作系统提供的进程、文件、网络和系统调用使用这些资源。

这也符合“不重复造轮子”的原则。

Linope 内核关注的是 Agent 特有的资源：

- memory；
- tool；
- sandbox；
- network。

这些资源不是硬件资源，而是 Agent 在运行时需要申请、隔离、共享和回收的能力资源。

Linope 资源管理的基本关系可以概括为：

代码块

```
需求 → 声明 → 实体 → 租约 → 句柄
```

能力首先声明自己需要什么资源。内核根据声明创建或取得资源实体，再为具体进程建立租约，并向进程发放句柄。

进程真正看到的是句柄、资源类型和调用参数，而不是内核内部的完整资源结构。

资源实体和进程账本也被分开：

- 资源实体保存实际运行状态；
- 租约管理谁正在使用资源；
- 进程账本记录某个 pid 持有哪些资源；
- 句柄用于检查调用者是否仍然拥有访问资格。

这种设计使得内核不需要为每一种新资源修改进程管理逻辑。只需要在资源契约中登记新的资源类型，就可以复用现有的实体、租约、句柄和账本机制。

内存：Agent 的上下文空间

如果将 token 视作 Agent OS 中的内存单位，那么模型的上下文窗口就类似于传统操作系统中的可用内存容量。

一次模型调用能够携带多少内容，取决于模型允许的上下文长度。上下文越长，模型能够看到的历史信息越多；上下文被占满之后，就必须进行裁剪、压缩，或者把任务交给其他进程处理。

不过，token 并不等于 Linope 中的 memory 本身。

更准确地说：

- memory 保存上下文内容；
- context 是从 memory 中投影出来、准备发送给模型的内容；
- token 是衡量这些内容占用多少上下文空间的单位；
- context limit 是模型能够接受的最大上下文长度。

因此，Linope 的 memory 更接近“上下文内存”，而不是数据库、向量数据库或永久记忆系统。

与以会话日志为中心的 Agent 框架对比

再次以 deepseek-harness 为例，它把一次 Agent 会话保存为追加式的 SessionEvent 日志。用户消息、模型消息和工具结果都成为事件。

之后，系统通过 surface 把能够进入模型上下文的事件投影成消息列表，再由 deriveMessages() 重建模型历史。

它的核心关系是：

代码块

```
SessionEvent 日志
      ↓
```

```
surface 投影
    ↓
deriveMessages()
    ↓
模型消息历史
```

上下文压缩时，系统会用摘要消息替换原来的消息范围。原始事件仍然保留在日志中，模型下一次看到的是替换后的 surface。

这种方式强调：

- 会话日志的持久化；
- 请求的可重放；
- 压缩过程的可追踪；
- 模型上下文的确定性重建。

Linope 关注的是另一个问题。

Linope 中的 memory 不再只是会话日志的另一种写法，而是拥有资源身份、访问权限、虚拟地址和生命周期的内核资源。

两种方式可以概括为：

代码块

```
以会话日志为中心：
事件日志 → 消息投影 → 完整上下文 → 压缩替换

Linope：
资源租约 → 指针空间 → selector 授权 → 按需读取
```

前者解决“如何可靠地保存和重建一次 Agent 会话”，后者解决“多个平级能力进程如何隔离、共享并访问上下文”。

Memory 是一种资源

在传统操作系统中，进程不会直接拥有整台计算机的物理内存，而是通过操作系统获得一段受管理的地址空间。

Linope 也采用了类似的思路。

memory 被注册为一种资源类型，它要求调用者提供一个 scope，例如：

代码块

```
memory=chat
memory=medical-session
```

```
3   memory=coding-task
```

这个 scope 是一份逻辑 memory 空间的名字。内核根据资源类型和身份生成 resource_id，再为进程建立租约和句柄。

模型本身并不天然拥有 memory。当前项目中，模型能力默认声明网络资源，必要时再通过 --with memory=chat 给它附加一份记忆资源。

这样设计以后：

- 一个模型可以没有 memory，只接受一次性输入；
- 一个模型可以拥有独立 memory；
- 多个模型可以明确共享同一份 memory；
- 上下文压缩能力也可以作为独立进程使用 memory。

内核不关心这份 memory 保存的是 Coding 对话、医疗咨询还是搜索任务。内核只负责建立资源、分配权限和管理生命周期。

指针：上下文访问的入口

Linope 的 memory 不是一段可以被进程直接读取的字典。进程必须先获得一个指向 memory 的指针。

在 kernel/scheduler/memory_space.py 中，memory 指针包含：

代码块

```
1   scope       memory 的名字
2   resource_id 底层资源实体
3   classes     允许访问的类别
4   handle      进程持有的句柄
```

因此，指针不只是一个地址，还包含访问范围。

例如：

代码块

```
1   scope = chat
2   classes = ("message", "artifact")
```

表示当前进程可以访问 chat 中的消息和工具结果，但不能访问 task 或 system。

内核会根据进程已经持有的资源授予，建立进程自己的指针空间。

进程即使知道某个 memory 的名字，也不能直接访问它。只有同时满足以下条件，访问才会通过：

1. 进程持有对应的 memory；
2. scope 在自己的指针空间中；
3. 目标类别属于允许范围；
4. 资源句柄仍然有效。

Memory 内部保存原始内容

一份新的 memory 在打开时，最初只包含一个地址空间信息：

```
代码块
1   address_space:
2       scope: chat
```

随着进程工作，它会逐渐形成一份字典，主要包含：

- items：实际保存的消息、任务和工具结果；
- next：不同类别的下一个编号；
- seq：整份 memory 的全局递增序号；
- system：系统提示词；
- prompt_files：已经加载过的提示文件。

一行内容通常具有以下信息：

- class：属于 message、task、artifact 还是 system；
- kind：属于 user、model、tool 还是 other；
- content：原始内容；
- time：写入时间；
- seq：在整份 memory 中的顺序；
- state：例如 asked、resulted、rejected 或 answered。

不同类型的内容会被放进不同类别：

- 用户输入和模型输出属于 message；
- 工具调用结果属于 artifact；
- 其他调度任务属于 task；
- 系统提示词属于 system。

例如：

```
代码块
```

```
message/0  → 用户问题
message/1  → 模型回答
artifact/0 → 工具返回结果
task/0     → 交给另一个进程的任务
```

这里的编号不是全局编号，message/0 和 artifact/0 可以同时存在，因为它们属于不同类别。

真正用于跨类别排序的是 seq。这样，memory 既可以按照类别访问，也可以按照时间顺序重新组织成模型消息。

系统提示词也不是简单地拼接到每次请求中。通过 write_prompt 写入的提示文件会保存在 prompt_files 中，并按照指定位置组合成一条 system 行。

相同名字的提示文件会被替换，而不会无限重复追加。

为什么需要每个进程自己的地址表

多个进程可以共享同一个 memory 实体，但不能因此共享同一套本地地址。

例如，进程 A 先加入 memory：

代码块

```
底层：
message/0 = 第一条消息

进程 A：
message/0 → message/0
```

之后进程 B 加入，进程 A 又写入一条消息：

代码块

```
底层：
message/0 = 第一条消息
message/1 = 第二条消息

进程 A：
message/0 → message/0
message/1 → message/1

进程 B：
message/0 → message/1
```

此时，A 的 message/1 和 B 的 message/0 指向同一条真实记录。

Linope 中的几个地址需要区分：

- address：进程看到的本地地址；
- real：memory 内部的真实记录名；
- scope：资源空间的名称；
- resource_id：内核内部的实体标识。

这和传统操作系统的虚拟地址空间比较接近：多个进程可以映射同一块底层资源，但各自使用的虚拟地址不必相同。

Memory 还有自己的地址空间

如果只有一份共享字典，那么多个进程直接使用 message/0、message/1，就会产生问题。

不同进程加入 memory 的时间不同，能够看到的类别也不同。如果所有进程直接使用底层编号，那么新加入的进程会自动看到之前所有记录，隔离就会被破坏。

因此，Linope 把 memory 分成两层：

第一层是 memory 实体，保存真正的内容；

第二层是进程自己的 pointer space，决定这个进程能够进入哪一份 memory，以及能够看到其中哪些类别。

例如：

代码块

```
message/0
message/0-10
artifact/0
chat/message/0
```

其中：

- message/0 是当前进程地址空间中的第 0 条消息；
- message/0-10 是当前进程能够看到的消息范围；
- chat/message/0 表示通过 scope 进入名为 chat 的 memory；
- scope 本身不是权限，只有进程确实拥有指向 chat 的 pointer，读取才会被允许。

模型如何使用指针

在页面模式下，模型不会一开始就获得完整历史消息。

memory_page() 会把当前回合直接放在页面中，而较早的内容只展示地址、类型和时间：

代码块

```
message/0
```

```
artifact/0
message/1
```

如果模型需要查看历史消息，就会产生一个 MemoryRead 请求。权限通过后才执行真正读取。

流程大致是：

代码块

```
模型请求 message/0-3
    ↓
authorize_selector()
    ↓
PointerSpace.require()
    ↓
memory_contents()
    ↓
生成下一页
```

页面上的地址只是“可以尝试读取的入口”，并不等于授权。真正的授权仍然掌握在调度器中。

这种按需读取方式符合一个重要原则：

上下文不是越多越好，而是要把对当前决策有用的信息送给模型。

历史内容不会无条件占满上下文。模型可以根据当前任务选择需要的地址，内核再检查它是否有权读取。

一次模型回合大致经过以下过程：

1. 用户问题写入 memory；
2. 调度器生成当前页面；
3. 模型选择读取历史、调用工具、询问其他模型，或者直接回答；
4. 调度器验证模型请求；
5. 执行通过的动作；
6. 把工具结果或其他进程的结果写回 memory；
7. 生成下一页；
8. 直到模型给出最终答案。

Memory 的读取不会直接启动工具或进程。读取仍然只是读取，只有模型明确请求调用工具、启动能力进程或询问另一个模型时，调度器才会执行相应动作。

上下文压缩是能力，不是 Memory 本身

什么时候触发压缩、压缩成功后是否替换原始上下文，属于宿主的策略。

这体现了 Linope 的一个基本取舍：

内核负责管理上下文资源和访问边界，压缩能力负责决定如何重写上下文。

内核不应该替某个 Agent 决定“哪些记忆值得保留”。

对于 Coding Agent，文件路径、编译错误和测试结果可能最重要；对于医疗顾问 Agent，病史、药物和时间顺序可能最重要。压缩规则应该属于能力，而不是写死在内核里。

Memory 的生命周期

Memory 的生命周期跟随资源租约，而不是跟随某个固定的父子进程树。

进程启动时：

1. 声明自己需要 memory；
2. 内核根据 scope 编译资源实体；
3. 资源池创建或取得 memory；
4. 为进程分配句柄；
5. 建立资源账本和地址空间；
6. 进程开始读写内容。

进程退出时：

1. 句柄被释放；
2. memory 的引用计数减少；
3. 如果仍有其他进程共享，memory 继续保留；
4. 如果引用计数归零，资源被卸载。

当前项目中的 ledger 只保存资源账本，例如：

- pid；
- 资源类型；
- resource_id；
- 句柄；
- 身份；
- 租约状态。

它不保存具体的消息正文。

也就是说，memory 的正文属于运行时资源，而不是账本文件中的永久数据。Linope 当前的 memory 更准确地说是一份受内核管理的运行时上下文内存，而不是面向长期知识保存的数据库。

如果未来需要保存跨会话的用户资料、项目知识或医疗档案，可以把数据库、向量库或文件系统作为新的能力资源，通过工具进程提供给 Agent。

总结

Linope 的设计可以概括为几个层次。

在能力层，模型、工具和上下文压缩器都是平级能力进程。

在进程层，每个能力进程拥有独立的生命周期、pid 和资源账本。

在资源层，内核通过资源实体、租约、句柄和引用计数管理 memory、tool、sandbox 和 network。

在上下文层，memory 保存原始内容，pointer space 管理访问权限，selector 描述读取范围，页面机制让模型按需获取历史。

在调度层，内核根据当前存活的进程、资源状态、上下文压力和并发情况完成请求路由。

Linope 与以会话日志为中心的 Agent 框架关注点不同。后者主要解决会话如何持久化、投影和重建；Linope 则进一步把 Agent 拆解为一组可以被内核隔离、共享和调度的能力进程。

因此，Linope 所提出的并不只是一个新的 Agent 框架，而是一种新的 Agent 开发范式：

Agent 不再是一个绑定了许多插件的单体会话，而是由内核管理的一组能力进程。模型负责规划和选择能力，内核负责进程、资源、权限和通信。